3DGEngine Shader Editor

Examples Guide

Build, preview, validate, and use shader graphs

Surface | Post Process | Particle | Unlit / UI

Updated for the current typed graph, parameter metadata, and Material Maker workflow

Contents and quick start

Five-minute workflow

- Open **Panels > Shader Editor** and choose **New**.
- Choose the shader **Domain** before building the graph. Domain conversion is supported, but incompatible domain-only nodes are removed after confirmation.
- Right-click empty graph space to add a node, or drag from a pin into empty space to open a compatible-node prompt.
- Connect typed output pins to compatible input pins on the domain output node.
- Create parameters for values that artists should edit in Material Maker. Configure their group, tooltip, range, step, and material visibility.
- Use **Compile** or **Auto Compile**, inspect the live preview, save the **.3dgshader**, then select it in Material Maker.

> **Important**
>
> Object Color comes from the scene object's Rendering Color. Use Parameter Color when a color must be stored and edited as part of a material.

1. Editor workflow and interface

The Shader Editor is an isolated authoring panel for typed shader graphs. It keeps the last valid GPU program active when a new compile fails, so an error does not destroy the previous preview.

Area	Purpose
Document toolbar	New, open selected, save, save as, revert, and duplicate shader assets.
Compile controls	Manual compile, Auto Compile, Apply, and Restore Last Valid.
Typed graph	Create, connect, select, move, duplicate, copy, paste, align, frame, and auto-layout nodes.
Selected Node	Rename nodes, edit constants, configure parameters, assign engine textures, and add comments.
Live Preview	Preview on sphere, cube, plane, or an imported static/skeletal model.
Generated Source	Inspect generated vertex and fragment GLSL with line numbers and search.
Diagnostics	Select compile or graph validation messages to focus the related node.

Graph navigation

- Middle-mouse drag pans the graph; the mouse wheel zooms around the pointer.
- Drag a pin onto another compatible pin to connect it. Drag a pin into empty space to add and connect a compatible node.
- Right-click a node for node-specific actions, including deletion. This avoids the scene-object Delete shortcut conflict.
- Use Frame or the minimap to recover nodes outside the visible graph area.
- Undo and Redo preserve graph edits, including safe domain conversion.

Preview controls

- Orbit yaw, pitch, and distance control the preview camera.
- Time of Day, Environment Rotation, and Light Intensity test a Surface shader under different lighting.
- Ground + Shadow helps judge contact and roughness. Bloom helps evaluate emissive values.
- Use Selected Model loads the currently selected engine-owned static or skeletal model into the isolated preview.

2. Domains and output contracts

Domain	Output	Use
Surface	Base Color, Emissive, Roughness, Metallic, Normal, Opacity, Alpha Cutoff, Clearcoat, Transmission, Subsurface, Sheen, Anisotropy, Displacement	Lit static and skeletal materials.
Post Process	Color	Full-screen effects that sample scene color, depth, normal, or velocity.
Particle	Color	Particle rendering using particle color, age, velocity, size, rotation, frame, and trail coordinates.
Unlit / UI	Color	HUD and unlit rendering independent of scene lighting.

Surface output guidance

Socket	Typical range	Meaning
Roughness	0 to 1	0 is sharp; 1 is broad and matte.
Metallic	0 or 1	Use intermediate values only for intentional blends.
Opacity	0 to 1	Requires a compatible blend mode.
Alpha Cutoff	0 to 1	Masked pixels below the threshold are discarded.
Emissive	0 and above	Values above 1 are useful with bloom.
Displacement	Small world/local amount	Offsets vertices along the local normal.

Domain conversion

Changing Domain opens a confirmation dialog. The editor rebuilds the canonical output sockets, preserves a compatible Color/Base Color connection, removes nodes exclusive to the old domain, and allows the entire conversion to be undone.

Best practice

Choose the final domain first. Convert only when the shader's role truly changes, then review every output connection before saving.

3. Nodes, parameters, and material controls

Node pins are typed. A link is accepted only when the source and destination are compatible. Float values may feed vector/color inputs as a replicated scalar, while Color and Vector4 are mutually compatible.

Category	Representative nodes
Inputs	UV, WorldPosition, LocalPosition, Normal, Tangent, ViewDirection, CameraPosition, ObjectColor, Time, DeltaTime
Parameters	Float, Int, Bool, Vector2, Vector3, Vector4, Color, Texture2D
Constants	ConstantFloat, ConstantVec2, ConstantColor
Math	Add, Subtract, Multiply, Divide, Min, Max, Clamp, Saturate, Power, SquareRoot, Absolute, Floor, Fraction, Modulo
Vector	Compose, Split, Swizzle, Dot, Cross, Normalize, Length, Reflect, Lerp
Texture	SampleTexture2D, NormalMapDecode, UVTransform, ChannelMask
Utility	OneMinus, Remap, Smoothstep, Fresnel, Noise, Comparison, Select

Parameter authoring

- Rename the parameter node with an artist-facing name such as **Edge Glow Strength**.
- Set a default value. Color uses a color picker; textures accept engine-owned **.3dgtex** assets.
- Set **Group** to organize Material Maker controls, for example Surface, Glow, or Distortion.
- Add a short **Tooltip** that explains the visible result rather than implementation details.
- Enable **Use Editor Range** for numeric parameters and set Minimum, Maximum, and Step.
- Disable **Visible in Materials** for internal parameters that should retain a default but not be artist-editable.

Uniform safety

Display names are converted to stable GLSL uniforms. The editor rejects names that collide after sanitization, such as 'Coat-Roughness' and 'Coat Roughness', and rejects engine-reserved uniforms.

4. From shader graph to material instance

A shader defines behavior and exposed defaults. A material stores the selected shader plus parameter overrides. Multiple materials can reuse one shader without duplicating the graph.

Recommended Content layout

```
Content/Assets/Shaders/MagicSurface.3dgshader
Content/Materials/MagicSurface_Blue.3dgmat
Content/TextureImages/MagicNoise.3dgtex
```

Material Maker procedure

- Save the graph as a **.3dgshader** inside Content.
- Open Material Maker and choose the shader from the searchable **Custom Shader** dropdown.
- Edit grouped parameter controls. Sliders use the range and step authored in the Shader Editor.
- Use Reset beside a parameter to restore the shader default.
- Use Reload Definition after changing the shader. Compatible overrides are preserved, new parameters receive defaults, and removed parameters are discarded.
- Save the material as **.3dgmat**, then assign it from the selected object's Rendering section.

Runtime parity

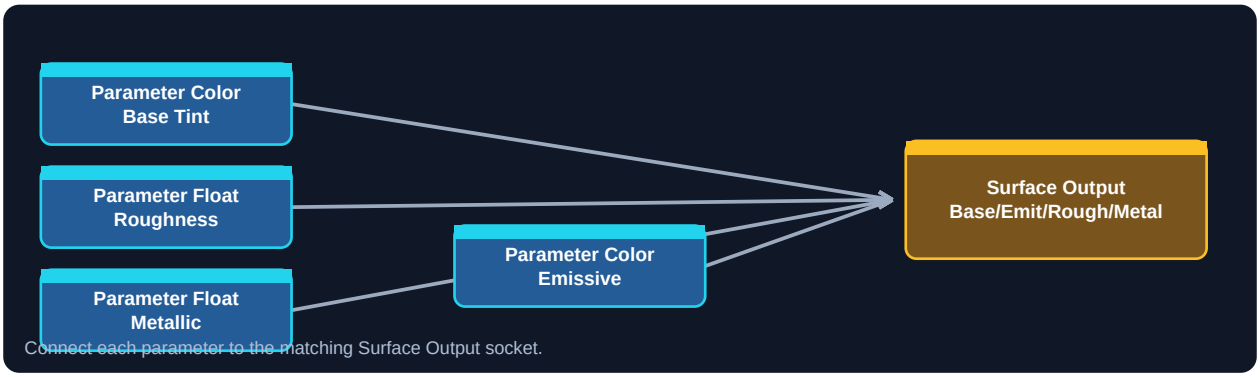
The same parameter and texture binding path is used by the Shader Editor preview, static meshes, skeletal meshes, particles, post-processing, HUD shaders, runtime-loaded materials, and scene material overrides. A value that previews correctly should therefore bind with the same type and uniform name at runtime.

Texture rule

Import source images as engine-owned .3dgtex assets before assigning them to Texture2D parameters. Color maps normally use sRGB; normal, roughness, metallic, AO, height, and packed data maps are linear data.

5. Example 1: configurable PBR surface

Goal: create a reusable lit shader whose base color, roughness, metallic response, and emissive color are controlled by Material Maker.



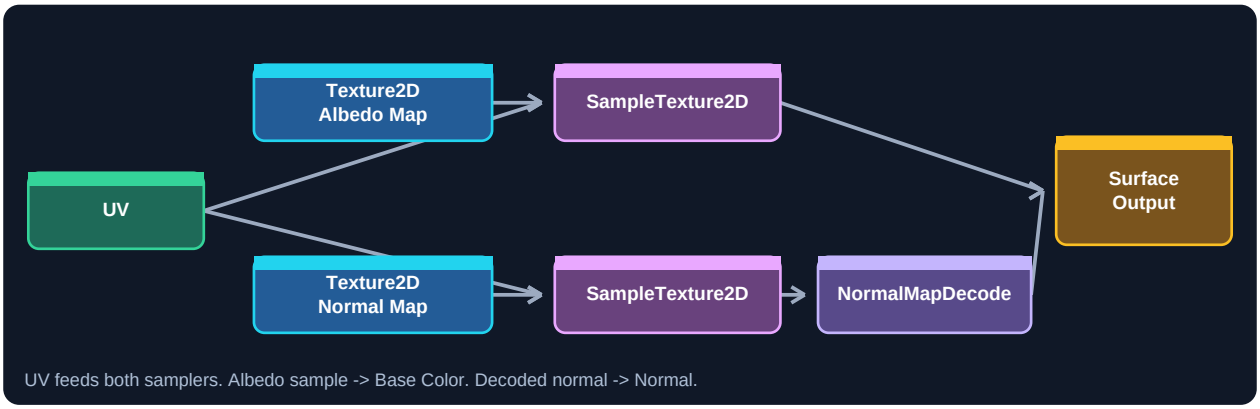
Build steps

- Add Parameter Color, name it **Base Tint**, default it to a neutral color, and connect it to Base Color.
- Add Parameter Float **Roughness**. Set range 0.04 to 1.0, step 0.01, group Surface, then connect it to Roughness.
- Add Parameter Float **Metallic**. Set range 0 to 1, step 1 for a clear dielectric/metal choice, then connect it to Metallic.
- Add Parameter Color **Emissive**, group Glow, and connect it to Emissive.
- Compile and compare the sphere under different Time of Day and Environment Rotation settings.

Parameter	Default	Suggested metadata
Base Tint	0.55, 0.18, 0.08, 1	Group Surface; tooltip: Base reflected color.
Roughness	0.45	Range 0.04 to 1; step 0.01.
Metallic	0	Range 0 to 1; step 1.
Emissive	0, 0, 0, 1	Group Glow; increase RGB above 1 through vector editing when strong bloom is needed.

6. Example 2: textured surface with normal mapping

Goal: sample engine-owned color and normal textures with shared UVs, decode the tangent-space normal, and expose roughness/metallic controls.



UV feeds both samplers. Albedo sample -> Base Color. Decoded normal -> Normal.

Build steps

- Import the source images into Content as .3dgtex assets.
- Add UV, two Parameter Texture2D nodes, and two SampleTexture2D nodes.
- Connect Albedo Map and UV to the first sampler; connect its Result to Base Color.
- Connect Normal Map and UV to the second sampler; connect Result to NormalMapDecode, then connect the decoded result to Normal.
- Add Float parameters for Roughness and Metallic and connect them directly to the corresponding output sockets.

Common mistake

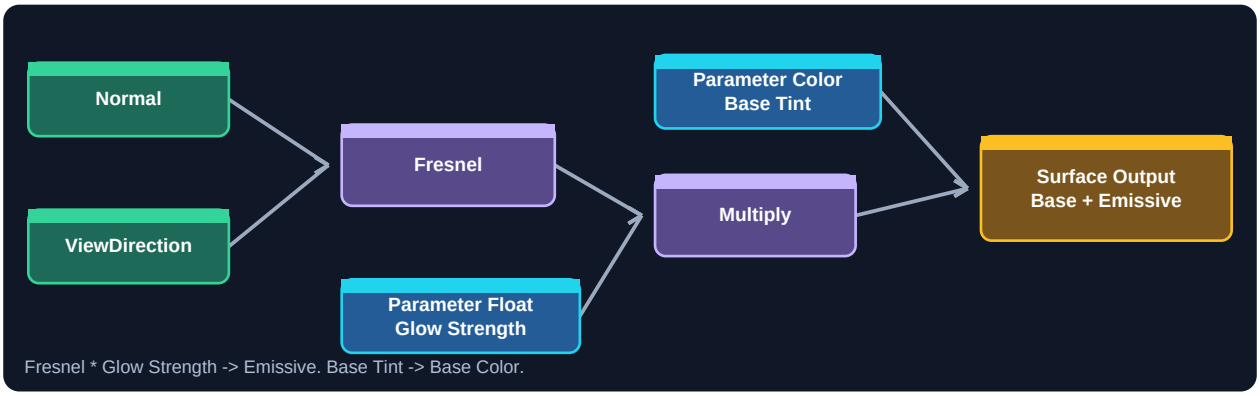
Do not connect the raw normal texture color directly to Normal. Use NormalMapDecode so the stored 0..1 RGB values become a normalized tangent-space direction.

Optional UV controls

Insert UVTransform between UV and both samplers. Use Vector2 parameters for Scale and Offset. Keeping both samplers on the same transformed UV prevents the color and normal detail from sliding apart.

7. Example 3: magical rim glow

Goal: produce an edge-brightening scalar with Fresnel and feed it into Emissive while keeping the surface's base color independently editable.



Build steps

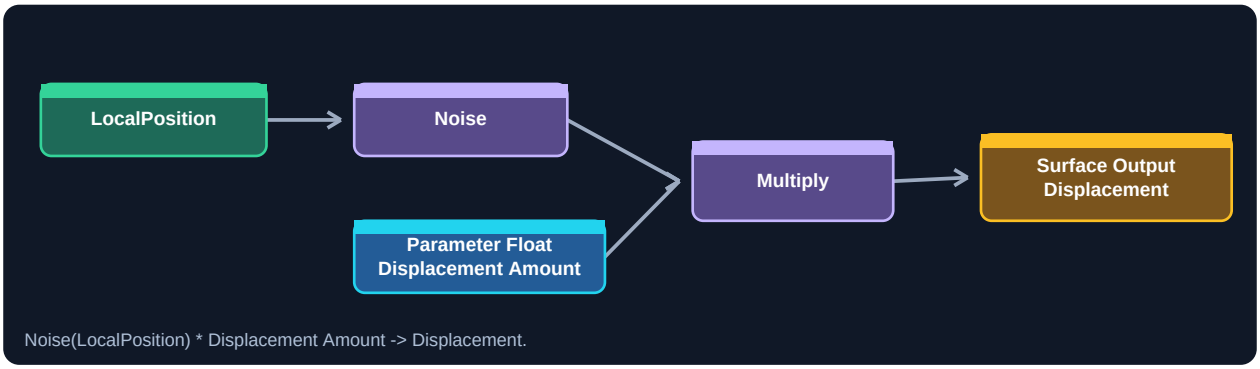
- Connect Normal and ViewDirection to Fresnel.
- Multiply Fresnel by Parameter Float **Glow Strength**.
- Connect the Multiply result to Emissive. A float feeding a color is replicated across RGB, producing a white rim.
- Connect Parameter Color **Base Tint** to Base Color and set Surface roughness separately.
- Enable Bloom in the preview. Start Glow Strength at 1 and raise it carefully.

Color limitation

The current typed Multiply node is scalar. For a colored rim, use the scalar emissive rim with the object's/material context, or author a future color-multiply node when that operation is added. Do not force incompatible color links.

8. Example 4: procedural surface displacement

Goal: offset Surface vertices along their local normal using procedural noise. This affects geometry, so mesh density determines how much detail can appear.



Build steps

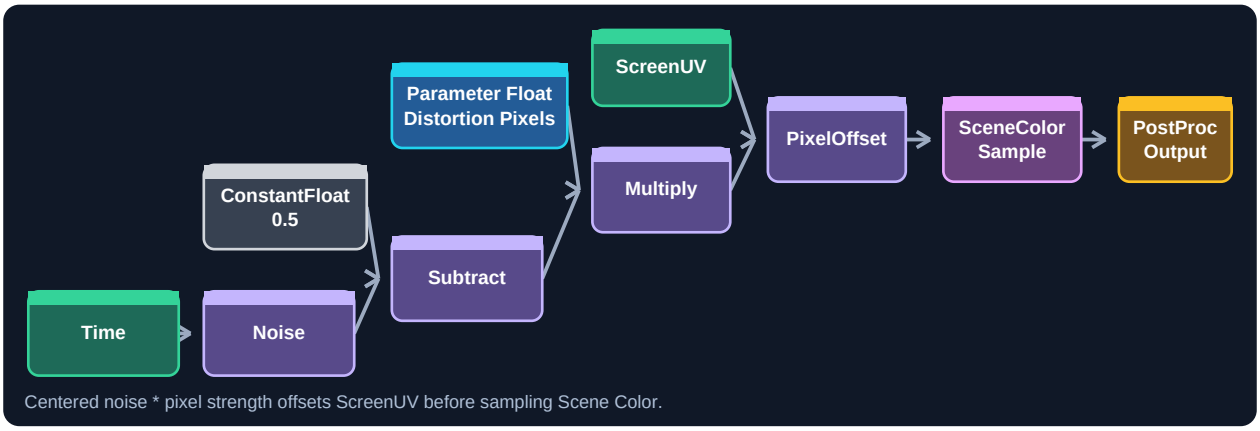
- Add LocalPosition and connect it to Noise. The scalar Noise result varies across the object.
- Add Parameter Float **Displacement Amount**, range 0 to 0.25, step 0.005.
- Multiply Noise by the amount and connect the result to Displacement.
- Preview first on a sufficiently subdivided imported model. A low-poly cube can only move its existing vertices.
- Keep displacement small enough that collision and silhouette expectations remain reasonable.

Runtime note

Surface displacement is generated in the vertex stage for both static and skinned variants. Fragment-only operations such as texture sampling resolve to a safe zero in the displacement evaluator.

9. Example 5: post-process screen distortion

Goal: offset scene sampling in pixel-sized units with procedural noise. The effect remains resolution-aware because PixelOffset multiplies by TexelSize internally.



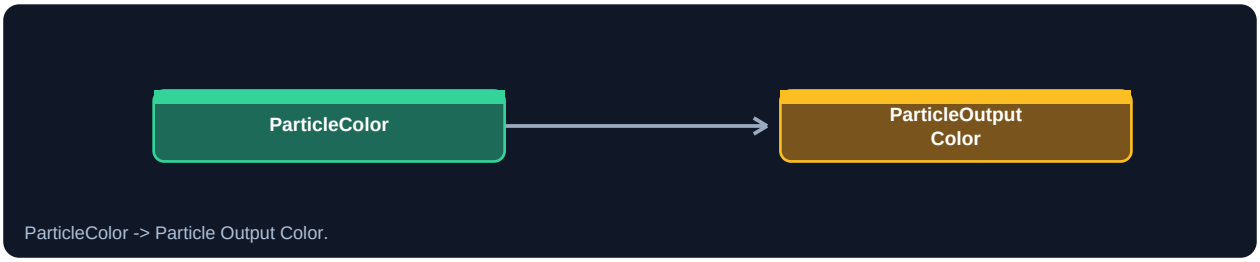
Centered noise * pixel strength offsets ScreenUV before sampling Scene Color.

Build steps

- Create a Post Process shader. Add ScreenUV, Time, Noise, ConstantFloat 0.5, Subtract, Multiply, PixelOffset, SceneColorSample, and the output.
- Feed Time into Noise. Subtract 0.5 so the offset can move in both directions.
- Multiply by Parameter Float **Distortion Pixels**, range 0 to 12, step 0.25.
- Connect ScreenUV to PixelOffset UV and the scaled noise to PixelOffset Pixels.
- Connect PixelOffset Result to SceneColorSample UV, then Result to Post Process Output Color.
- Add the saved shader to World Settings' post-process stack and tune the parameter from its effect controls.

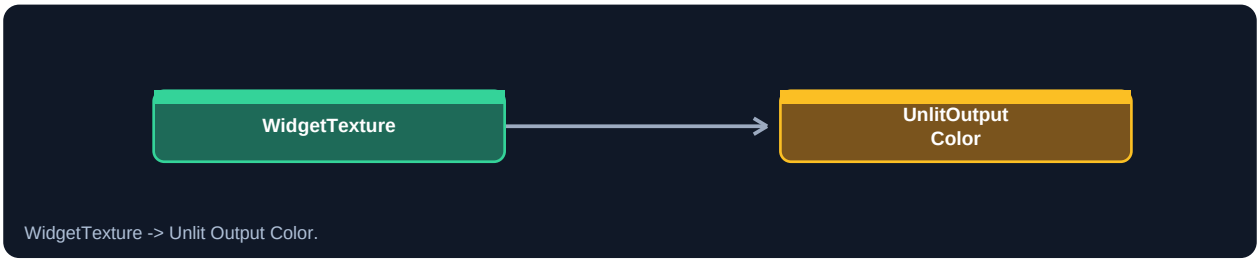
10. Examples 6 and 7: Particle and Unlit/UI

Example 6: particle color pass-through



- Create a Particle-domain shader and connect ParticleColor directly to Color.
- Save it and select it in the Particle Editor's render-stage custom shader control.
- The particle emitter continues to control per-particle color and alpha; the shader receives the same runtime particle attributes used by its preview contract.
- ParticleAge / NormalizedLifetime, ParticleVelocity, ParticleSize, ParticleRotation, ParticleFrame, and TrailCoordinates are available for future scalar/vector effects.

Example 7: HUD texture shader



- Create an Unlit-domain shader and connect WidgetTexture to Color.
- Save it, open the HUD Editor, select an image/panel widget, and assign the shader where the widget custom shader is supported.
- WidgetTexture automatically resolves to the widget image when one is active, or white when no texture is supplied.
- WidgetColor, WidgetUV, ClipMask, and SignedDistance are available for other UI graph designs.

Domain rule

Particle-only and Widget-only nodes are rejected in other domains. If a saved graph reports a domain-specific input error, change the graph domain or replace the incompatible node.

11. Saving, applying, and runtime behavior

Saving the shader writes an engine-owned .3dgshader asset with a stable asset identity, graph data, typed parameters, parameter metadata, and texture dependencies. Referencing systems resolve the identity first and retain a fallback path for recovery.

Surface shader deployment

- Save the shader under Content.
- Create or open a .3dgmat in Material Maker.
- Choose the Surface shader, edit overrides, and save the material.
- Select a scene object and assign the material in Rendering.
- Use a prefab or Character Asset when the configured mesh/material combination should be reusable.

Other domains

Domain	Where to use the saved shader
Post Process	World Settings post-process shader stack.
Particle	Particle Editor render-stage custom shader.
Unlit / UI	HUD/custom UI rendering path.

Hot reload and fallback

RuntimeShaderManager caches programs by asset path and variant. Successful compiles replace the cached program. A failed hot reload retains the previous valid program. A first-time failure receives a visible domain fallback rather than a null shader.

Static and skeletal parity

Surface graphs generate both static and skinned variants. Parameter values, textures, camera/time inputs, object color, and primary light inputs are uploaded consistently in both rendering paths.

12. Diagnostics and troubleshooting

Symptom	Likely cause	Fix
Graph will not connect	Pin types are incompatible or the link would create a cycle.	Use the compatible-node prompt; insert a supported conversion/math node.
Compile error	Missing required output, illegal domain node, malformed graph, or generated GLSL error.	Select the diagnostic to focus its node; repair the first error, then compile again.
Parameter missing in Material Maker	Visible in Materials is off, shader definition is stale, or the shader was not saved.	Save the shader, enable visibility, and choose Reload Definition.
Material appears black	Very low base color, invalid normal, no useful lighting, missing texture, or fallback program.	Test a constant bright Base Color, inspect diagnostics, verify .3dgtex assets, and rotate preview lighting.
Texture has no effect	Texture parameter is not connected through SampleTexture2D, or the material holds an old definition.	Connect Texture2D + UV to a sampler and reload the material definition.
Normal looks inverted	Raw normal color was connected directly or the source is not a normal map.	Use NormalMapDecode and a linear-data .3dgtex normal texture.
Post effect samples outside screen	Pixel offset is too large or UV math is not resolution-aware.	Use PixelOffset with a modest pixel range instead of adding raw UV values.
Displacement shows little detail	The mesh has too few vertices.	Preview on a denser imported mesh or reduce the expected frequency.
Uniform collision error	Two display names sanitize to the same GLSL name, or a name is engine-reserved.	Rename one parameter using a clearly distinct identifier.

Safe recovery sequence

- Do not delete the graph. Restore Last Valid to keep a working preview.
- Read Graph Validation before GPU compile diagnostics; structural errors often cause secondary GLSL errors.
- Select the first diagnostic and inspect the focused node and its incoming links.
- Compile manually after the correction. Save only after the expected preview returns.

13. Node and production checklist

Before saving the shader

- The intended domain is selected and the matching output node exists.
- Every required input is connected and there are no graph validation errors.
- Parameter names are unique, descriptive, and free of uniform collisions.
- Artist-facing parameters have groups, tooltips, useful defaults, and safe ranges.
- Internal parameters are hidden from materials when appropriate.
- Texture parameters use engine-owned assets and the correct color/data interpretation.
- The shader compiles and previews on the most relevant shape or imported model.
- Surface shaders have been checked under multiple lighting angles and intensities.

Before shipping a material

- Reload Definition once after the final shader save.
- Review every override and reset values that should follow shader defaults.
- Verify static and skeletal usage when the material can appear on both.
- Confirm masked/transparent behavior uses the correct blend mode and output values.
- Check the built player, not only the editor preview.

Quick reference

Need	Start with
Editable color	Parameter Color
Editable scalar	Parameter Float with range/step
Color texture	Parameter Texture2D + UV + SampleTexture2D
Normal texture	Texture sample + NormalMapDecode
Edge glow	Normal + ViewDirection + Fresnel
Vertex offset	Position/noise math -> Surface Displacement
Resolution-aware screen offset	PixelOffset -> SceneColorSample
HUD image	WidgetTexture -> Unlit Output
Particle tint	ParticleColor -> Particle Output

End of guide

Use these examples as small verified building blocks. Duplicate a working shader before experimenting, change one graph section at a time, and keep Auto Compile enabled while authoring.